Примитивы. Объекты

Занятие 5

Ход занятия

1. Примитивы.
2. Объекты.
3. Практика.

Типы данных

В JavaScript есть 7 примитивных типов данных:

- string
- number
- boolean
- null
- undefined
- bigint
- symbol

Объекты

Объект - это сложная структура данных, представляемая как неупорядоченная коллекция свойств и методов.

Свойство - это часть объекта которая состоит из имени и значения.

```
let user = {
    name: “Ivan”,
    age: “18”,
}
```

Объекты

Основные способы создания объекта:

- let user = new Object ();
- let animal = {};

Основными операциями при работе с объектами являются:

- добавление свойств.
- обращение к свойствам.
- изменение свойств.
- удаление свойств.

Добавление свойств объекта

```
let user = {};

user.name = “Ivan”;

user[“age”] = 18;
```

```
let value = “name”;

let people = {

    [value] : “Ivan”,

};

console.log( people.name ); //
Выведет Ivan
```

Обращение к свойствам объекта

```
let user = {
    name : “Ivan”;
    min-age : 18;
};
console.log(user.name); // Выведет Ivan
console.log(user[“min-age”]); // Выведет 18
```

Изменение свойств объекта

```
const user = {
    name : “Ivan”;
    age : 18;
};
user.name = “Anton”;
console.log(user.name); // Выведет Anton
```

Удаление свойств объекта

```
let user = {
        name : “Ivan”;
        age : 18;
};
 console.log(user.name); // Выведет Ivan
delete user.name;
 console.log(user.name); // Выведет undefined
```

Методы объекта

Метод - это свойство представленное в виде функции.

```
let user = {

    sayName: function () {
        console.log("Ivan")

    }

};

user.sayName(); // Выведет Ivan
```

Практика

- Создайте объект myFavoriteFilm, описывающий ваш любимый фильм. Объект должен содержать свойства с названием фильма, с датой выпуска, именем режиссера и страной выпуска.
- Добавить свойство содержащее значение выручки фильма в прокате.
- Добавить метод, который который будет выводить название фильма в консоль.
- Удалить свойство содержащее год выпуска.
- Вывести в консоль объект myFavoriteFilm и проанализировать его структуру.

Примитивы как объект

- Объекты «тяжелее» примитивов, занимают больше памяти.
- Но создателю JavaScript хотелось создать удобный доступ к методам и они должны были быть легкими и быстрыми.
- Выбранное решение: каждый примитив получил свой собственный «объект-обёртку»

Примитивы как объект

- string
- number
- boolean
- symbol
- null*
- undefined*

*Являются исключениями

Как это работает

- Примитив всегда остаются примитивами
- Язык позволяет осуществлять доступ к методам и свойствам строк
- Для этого создается специальный «объект-обёртка», который предоставляет нужный функционал, а после удаляется.

Так же каждый примитив имеет свой собственный «объект-обёртку», которые называются: String, Number, Boolean и Symbol. Таким образом, они имеют разный набор методов.

Пример использования

```
let str  = “Hello Ivan”;

console.log(str.toUpperCase());  // Выведет HELLO IVAN


let number = 1.2345;

console.log(number.toFixed(2));  // Выведет 1.23
```

Итого

- Формально эти методы работают с помощью временных объектов, но движки JavaScript внутренне очень хорошо оптимизируют этот процесс, так что их вызов не требует много ресурсов.
- Все примитивы, кроме null и undefined, имеют объект-обертку.
- new Number("123") и Number("123") разные. Первый это создание объекта, второй - приведение к числовому типу.

Числа (способы записи)*

В JavaScript можно использовать букву "e", чтобы сократить запись числа.

```
4.100.000.000 == 4.1e9 // 4.1 миллиардов
```

Другими словами, "e" производит операцию умножения числа на 1 с указанным количеством нулей.

```
0.000002 == 2e-6 // шесть нулей, слева от 1
```

Отрицательное число после "e" подразумевает деление на 1 с указанным количеством нулей.

Функции parseInt и parseFloat*

```
console.log( +'200px' ); // NaN

console.log( parseInt('200px') );  // 200

console.log( parseFloat('12.5em') ); // 12.5

console.log( parseInt('a555') ); // NaN, на первом символе происходит остановка чтения
```

Функция parseInt имеет необязательный второй параметр base. Он определяет систему счисления.

Функции isNaN и isFinite*

isNaN(value) преобразует значение в число и проверяет является ли оно NaN

```
console.log( isNaN(NaN) ); // true

console.log( isNaN("str") ); // true
```

isFinite(value) преобразует аргумент в число и возвращает true, если оно является обычным числом, т.е. не NaN/Infinity/-Infinity

```
console.log( isFinite("15") ); // true

console.log( isFinite("str") ); // false, потому что специальное значение: NaN

console.log( isFinite(Infinity) ); // false
```

Метод toString()

Метод num.toString(base) возвращает строковое представление числа num в системе счисления base.

```
let num = 255;

console.log( num.toString() );  // "255"

console.log( num.toString(16) );  // ff

console.log( num.toString(2) );   // 11111111
```

Объект Math

В JavaScript встроен объект Math который содержит различные

математические функции и константы.

Библиотека маленькая, но содержит всё самое важное.

* developer.mozilla.org

Связанные темы

Стандартные встроенные объекты

Math

Свойства

- Math.E
- Math.LN10
- Math.LN2
- Math.LOG10E
- Math.LOG2E
- Math.PI
- Math.SQRT1_2
- Math.SQRT2

Методы

- Math.abs()
- Math.acos()
- Math.acosh()
- Math.asin()
- Math.asinh()
- Math.atan()
- Math.atan2()
- Math.atanh()
- Math.cbrt()
- Math.ceil()
- Math.clz32()
- Math.cos()
- Math.cosh()
- Math.exp()
- Math.expm1()
- Math.floor()

Округление

Math.round - Округление до ближайшего целого

```
let num = 12.36;

console.log( Math.round(num) ); // "12"

console.log( num.toFixed(1) ); // "12.3" результатом toFixed является строка (+)
```

Math.floor - Округление в меньшую сторону

Math.ceil - Округление в большую сторону

Другие функции

Math.random() - Возвращает псевдослучайное число в диапазоне от 0 (включительно) до 1 (но не включая 1).

```
console.log( Math.random() ); // 0.1234567894322
```

Math.max(a, b, c...) / Math.min(a, b, c...) Возвращает наибольшее/наименьшее число из перечисленных аргументов.

```
console.log( Math.max(3, 5, -10, 0, 1) ); // 5
```

Итог

- Будьте очень осторожны при операциях с дробями, используйте округление.
- По математическим функциям смотрите Math.
- ParseInt \ ParseFloat - работают иначе +.
- toFixed(2) возвращает строку.

Строки

- Внутренний формат для строк — всегда UTF-16 вне зависимости от кодировки страницы.
- Содержимое строки в JavaScript нельзя изменить. Нельзя взять символ посередине и заменить его. Как только строка создана — она такая навсегда.

Спецсимволы

\n - Перевод строки

\r - Возврат каретки

\\ - Обратный слеш (\" экранирование)

\t - Знак табуляции

\u00A9 - Символ в кодировке UTF-16 формата \uXXXX, в данном случае юникодное представление знака копирайта ©

Свойства строк

Свойство length содержит длину строки.

```
console.log( `Ivan`.length ); // 4
```

Получить символ, который занимает позицию pos, можно с помощью квадратных скобок: [pos]

```
console.log( `Ivan`[0] ); // I если нет такой позиции, то undefined
```

Содержимое строки в JavaScript нельзя изменить.

```
let str = 'Hi';

str[0] = 'h';
```

Методы

```
console.log( 'Ivan'.toUpperCase() ); // IVAN

console.log( 'Ivan'.toLowerCase() ); // ivan

console.log( 'Ivan'[0].toLowerCase() ); // 'i'

console.log( ' Ivan '.trim() ); //убрать пробелы сначала и в конце строки
```

* developer.mozilla.org

Связанные темы

Стандартные встроенные объекты

String

Свойства

- String.prototype
- String.length

Методы

- String.fromCharCode()
- String.fromCodePoint()
- String.prototype.anchor()
- String.prototype.big()
- String.prototype.blink()
- String.prototype.bold()
- String.prototype.charAt()
- String.prototype.charCodeAt()
- String.prototype.codePointAt()
- String.prototype.concat()
- String.prototype.endsWith()
- String.prototype.fixed()
- String.prototype.fontcolor()
- String.prototype.fontsize()
- String.prototype.includes()
- String.prototype.indexOf()
- String.prototype.italics()
- String.prototype.lastIndexOf()
- String.prototype.link()
- String.prototype.localeCompare()

Сравнение строк

Строчные буквы больше заглавных

```
console.log( 'z' > 'Z' ); // true
```

Строки кодируются в UTF-16 Таким образом, у любого символа есть соответствующий код. Получить код символа str.codePointAt(pos).

```
console.log( "z".codePointAt(0) ); // 122
console.log( "Z".codePointAt(0) ); // 90
```

Поиск подстроки

str.indexOf(substr, pos) - Он ищет подстроку substr в строке str, начиная с позиции pos (необяз), и возвращает позицию, на которой располагается совпадение, либо -1 при отсутствии совпадений.

```
let str = 'Widget with id';

console.log( str.indexOf('id', 2) ) // 12
```

Чтобы найти все вхождения подстроки, нужно запустить indexOf в цикле.

Если нужно только вхождение*

str.includes(find, pos) - возвращает true, если в строке str есть подстрока substr, начиная с pos (необяз) либо false, если нет.

```
console.log( “Widget with id".includes("Widget") ); // true

console.log( "Widget with id".includes("ololo") ); // false
```

Получение подстроки*

.slice(start , end) - Возвращает часть строки от start до (не включая) end.

.substr(start, length) - Возвращает часть строки от start длины length.

.substring(start , end) - Возвращает часть строки между start и end.

```
let str = "stringify";

console.log( str.slice(0, 5) ); // вернет 'strin', символы от 0 до 5 (не включая 5)
```

Итог

- Есть три типа кавычек. (‘’, “ ”, ` `)
- Строки в JavaScript кодируются в UTF-16 у каждого символа есть код
- Для получения символа используйте []
- Для получения подстроки используйте slice, а вхождения indexOf и includes
- Содержимое строки в JavaScript нельзя изменить, но можно из нее создать новую.

Практика

- Реализовать функцию которая будет принимать числовой диапазон в качестве параметров [min, max] и будет возвращать случайное целое число из данного диапазона.
- Реализовать функцию которая будет определять, в каком регистре записан n элемент переданной строки, если в верхнем то вернуть true, в противном случае вернуть false.
- Реализовать функцию которая заменяет в строке str, все вхождения подстроки find, на подстроку replace.

Дополнительная информация

...

Неточные вычисления

Проблема:

```
console.log( 0.1 + 0.2 ); // 0.30000000000000004

console.log( 0.1 + 0.2 === 0.3 ); // false
```

Причины:

В JavaScript нет возможности для хранения точных значений 0.1 или 0.2, используя двоичную систему, точно также, как нет возможности хранить одну третью в десятичной системе счисления.

Неточные вычисления

Решение проблемы:

Наиболее надежный способ — это округлить результат:

```
let sum = 0.1 + 0.2;

console.log( +sum.toFixed(2) ); // 0.3


console.log( 9999999999999999 ); // покажет 10000000000000000
```